



Visvesvaraya Technological University, Belagavi

P E S Institute of Technology and Management, Shivamogga.



Department of Electronics and Communication Engineering.

“DESIGN, VERIFICATION AND TESTING OF A NON-VOLATILE (EEPROM) USING VERILOG HDL ” “FLASH MEMORY”

Presentation by

Mr. PRATHAM N RAO	USN : 4PM23EC074
Mr. SHREYASH BV	USN : 4PM23EC099
Mr. SUMANTH SG	USN : 4PM23EC107
Mr. VINAYAK SAVADI	USN : 4PM23EC121

5th Semester, Dept. of E&CE,
PESITM, Shivamogga.

Under the Guidance of
Mrs.Yashaswini N G
Assistant Professor
Dept. of E&CE, PESITM, Shivamogga.

Contents

- ❖ **Introduction**
- ❖ **Literature Survey**
- ❖ **Problem Statement**
- ❖ **Objectives**
- ❖ **Methodology**
- ❖ **Expected Outcome**
- ❖ **References**

Introduction

- Importance of NVM and EEPROM: Non-Volatile Memory (NVM) enables persistent data storage; EEPROM is especially valuable for its reliability and byte-level access, making it ideal for configuration and boot data.
- Challenges in Current EEPROM Modelling: Existing Verilog EEPROM models are often proprietary or too basic, lacking realism and flexibility for early-stage design and verification.
- Impact of Inadequate Verification: Memory subsystem failures can delay development and increase costs, highlighting the need for accurate pre-silicon verification.
- Project Objective: Design and implement a fully synthesizable, reliable, and reusable EEPROM IP core in Verilog.
- Verification Focus: Develop a thorough testbench to verify read/write functionality and timing behaviour, ensuring robustness before hardware rollout.

Literature Survey

Sr. No	Reference (in IEEE format)	Observations/Comments
1	Bez, E. Camerlenghi, A. Modelli, and A. Visconti, "Introduction to Flash Memory," in <i>Proceedings of the IEEE</i> , vol. 91, no. 4, pp. 489-502, Apr. 2003, doi: 10.1109/JPROC.2003.811702.	1. This paper provides an "Introduction to Flash Memory". 2. It is a foundational text for understanding non-volatile memory.
2	M. A. A. Sanvido, F. R. Chu, A. Kulkarni, and R. Selinger, "NAND Flash Memory and Its Role in Storage Architectures," in <i>Proceedings of the IEEE</i> , vol. 96, no. 11, pp. 1864-1874, Nov. 2008, doi: 10.1109/JPROC.2008.2004319.	1. (NAND FLASH MEMORY) It helps to understand different memory technologies and their role in storage architectures.
3	M. P. Kumar, A. K. Gupta, G. J. Rao, K. S. Kumar, and P. K. Vani, "An Efficient Flash Memory Devices," <i>2019 IEEE International Conference on Intelligent Systems and Green Technology (ICISGT)</i> , Visakhapatnam, India, 2019, pp. 49-52, doi: 10.1109/ICISGT44072.2019.00026.	1.This paper discusses "An Efficient Flash Memory Devices". 2.It provides insights into creating an efficient memory device.
4	P. Pavan, R. Bez, P. Olivo, and E. Zaroni, "Flash memory cells—an overview," in <i>Proceedings of the IEEE</i> , vol. 85, no. 8, pp. 1248-1271, Aug. 1997, doi: 10.1109/5.622505.	1.This work gives an "overview" of "Flash memory cells".
5	J. Bergeron, <i>Writing Testbenches: Functional Verification of HDL Models</i> , Second Edition, Springer, 2003.	1. This book is a guide for "Writing Testbenches".
6	1. M. S. Wagh and H. M. Pande, "FPGA implementation of memory interface logic for system on chip," in <i>2015 International Conference on Pervasive Computing (ICPC)</i> , Pune, India, 2015, pp. 1-4, doi: 10.1109/PERVASIVE.2015.7087224.	1.This paper discusses FPGA implementation of "memory interface logic". 2.It is relevant for ensuring the EEPROM model is "feasible for hardware synthesis"

Problem Statement

- In modern System-on-Chip (SoC) designs, **pre-silicon verification of non-volatile memories** like EEPROM is a critical step. However, there is a **lack of robust, synthesizable Verilog models for EEPROM** that can be effectively integrated into the design flow. Designers are forced to rely either on costly proprietary IP cores, which restrict flexibility, or on overly simplified behavioral models, which fail to capture real-world timing and bugs.
- This leads to **increased risk of post-silicon failures**, longer design cycles, and higher project costs.

Objectives

- To design:** The ROM module implements a parameterized synchronous ROM whose contents are initialized from a HEX file using \$readmemh. The output data updates one clock cycle after the address becomes stable.
- To test:** The testbench (tb) module verifies the ROM by generating a clock signal and using a synchronous task to sequentially read all memory locations, displaying the results for verification.
- To decode the error:** Errors are identified by comparing the displayed initial memory contents and the final read data output against the expected values, or by analyzing the generated wave.vcd waveform file.
- To implement:** The implementation is executed via a Bash script that uses iverilog to compile the Verilog files and vvp to run the simulation within the EDA Playground environment.

Block diagram



Fig 1: FLOW DIAGRAM

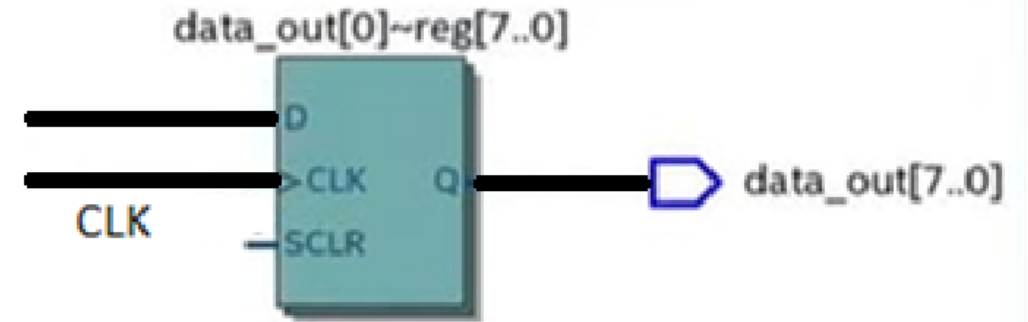


Fig 2: SCHEMATIC BLOCK DIAGRAM

IO Pins	Description
clk	Positive edge active clock
address_rd[3:0]	4bit address bus for read
data_out[7:0]	8bit data output bus

Fig 3: I/O PINS

Output



Fig 4: OUTPUT WAVEFORM

```
1000 ROM content:
3000 address = 0, data_rd = 0
5000 address = 1, data_rd = 11
7000 address = 2, data_rd = 22
9000 address = 3, data_rd = 33
11000 address = 4, data_rd = 44
13000 address = 5, data_rd = 55
15000 address = 6, data_rd = 66
17000 address = 7, data_rd = 77
19000 address = 8, data_rd = 88
21000 address = 9, data_rd = 99
23000 address = 10, data_rd = aa
25000 address = 11, data_rd = bb
27000 address = 12, data_rd = cc
29000 address = 13, data_rd = dd
31000 address = 14, data_rd = ee
33000 address = 15, data_rd = ff
testbench.sv:42: $finish called at 72600 (1ps)
Finding VCD file...
./wave.vcd
[2025-10-29 07:21:11 UTC] Opening EPWave...
Done
```

Fig 5: OUTPUT DATA



```
3 module rom
4 # (
5     parameter WIDTH = 8,
6     parameter DEPTH = 16,
7     parameter DEPTH_LOG = $clog2(DEPTH)
8 )
9 (
10     input  wire                clk,
11     input  wire [DEPTH_LOG-1:0] addr_rd,
12     output reg [WIDTH-1:0]      data_out
13 );
14
15     reg [WIDTH-1:0] mem [0:DEPTH-1];
16
17     initial begin
18         // Use only the basename when running in EDA Playground or most online
tools
19         // If you run locally from another directory, use an absolute path
instead.
20         $readmemh("rom_init.hex", mem, 0, DEPTH-1);
21         // quick verification printed at simulation start
22         $display("After $readmemh: mem[0]=%0h mem[1]=%0h mem[2]=%0h mem[3]=%0h
mem[0], mem[1], mem[2], mem[3]);
23     end
24
25     always @(posedge clk) begin
26         data_out <= mem[addr_rd];
27     end
28
29 endmodule
30
```

testbench.svrom_init.hexrun.bash✖✚

```
1 `timescale 1ns / 1ps
2
3 module tb();
4
5 localparam WIDTH      = 8;
6 localparam DEPTH      = 16;
7 localparam DEPTH_LOG  = $clog2(DEPTH);
8
9 reg  clk = 0;
10 reg  [DEPTH_LOG-1:0] addr_rd;
11 wire [WIDTH-1:0]    data_rd;
12
13 integer i;
14
15 // Instantiate the ROM
16 rom #(
17     .WIDTH(WIDTH),
18     .DEPTH DEPTH
19 ) ROM0 (
20     .clk(clk),
21     .addr_rd(addr_rd),
22     .data_out(data_rd)
23 );
24
25 // Clock generation: 1 ns period => toggle every 0.5 ns
26 always #0.5 clk = ~clk;
27
28 initial begin
29     // optional VCD for waveform viewing in tools that support it
30     $dumpfile("wave.vcd");
31     $dumpvars(0, tb);
32
33     #1;
34     $display("%0t ROM content:", $time);
35
36     // read every address (synchronous read: present address, wait one rising edge)
37     for (i = 0; i < DEPTH; i = i + 1) begin
38         read_data(i[DEPTH_LOG-1:0]);
39     end
40
41     #40;
42     $finish;
43 end
44
45 // task: apply address and display read value
46 task read_data(input [DEPTH_LOG-1:0] address_in);
47 begin
48     @(posedge clk); // wait edge before driving address (synchronous timing)
49     addr_rd = address_in;
50     @(posedge clk); // wait one clock for data_out to update
51     #0.1;
52     $display("%0t address = %2d, data_rd = %0h", $time, addr_rd, data_rd);
53 end
54 endtask
55
56 endmodule
```

testbench.svrom_init.hexrun.bash✖✚

```
1 00
2 11
3 22
4 33
5 44
6 55
7 66
8 77
9 88
10 99
11 AA
12 BB
13 CC
14 DD
15 EE
16 FF
```

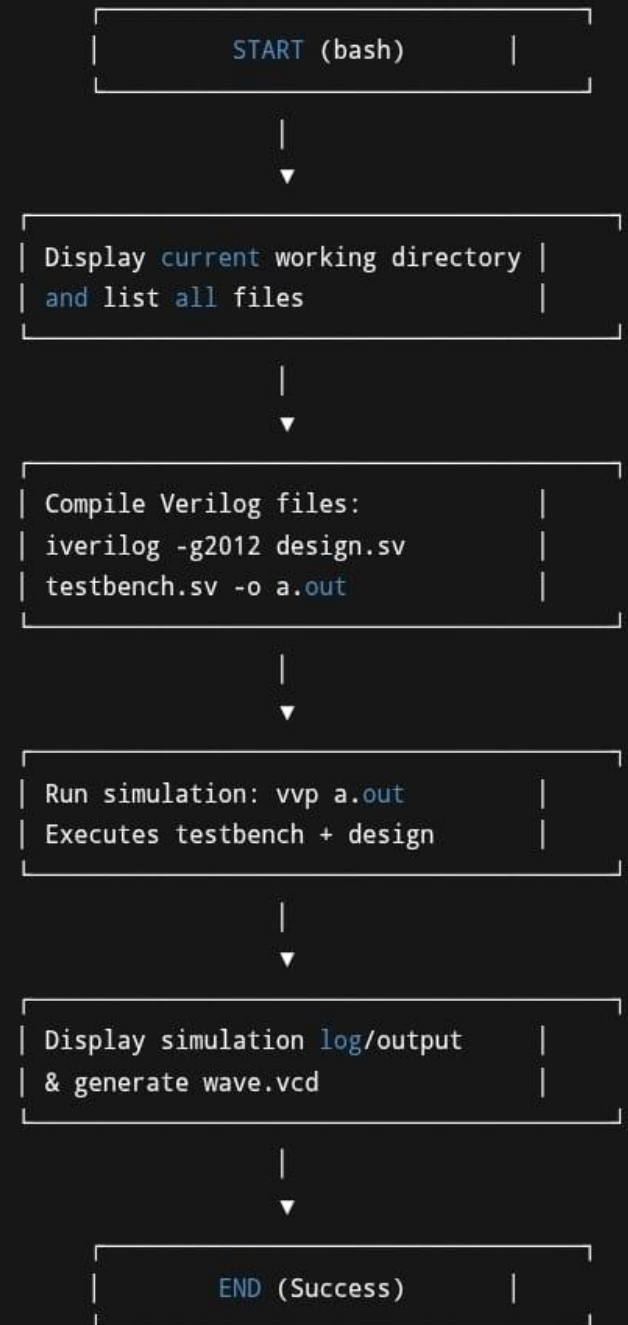
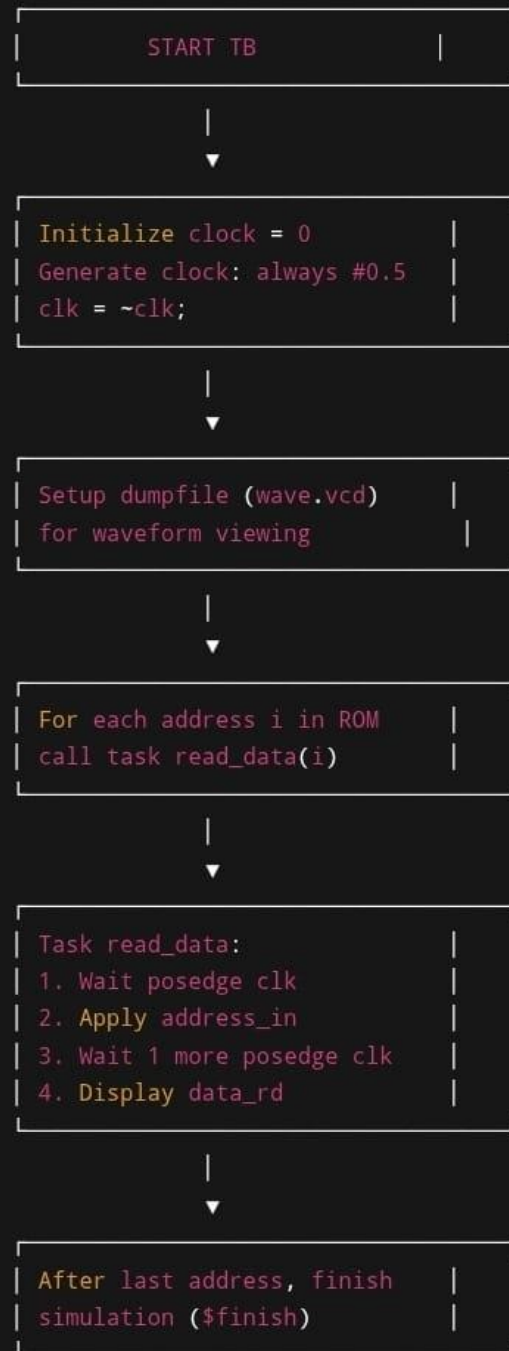
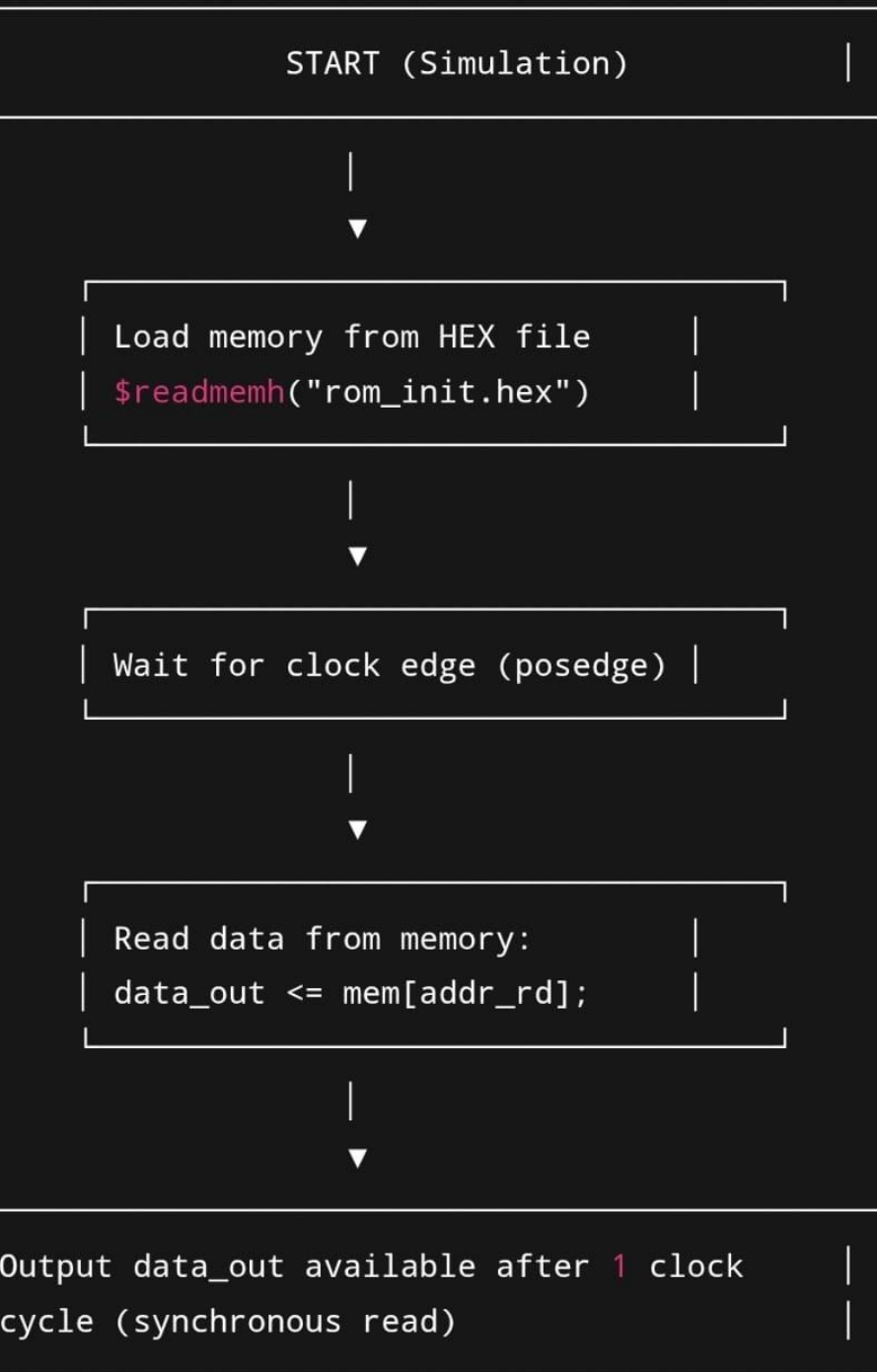
testbench.sv

rom_init.hex

run.bash



```
1 #!/usr/bin/env bash
2 set -euo pipefail
3
4 echo "PWD: $(pwd)"
5 echo "Files in cwd:"
6 ls -la
7
8 # compile
9 iverilog -Wall -g2012 design.sv testbench.sv -o a.out
10
11 # run
12 vvp a.out
```



Methodology

- **Fundamental Operating Principle** The core mechanism for programming and erasing flash memory cells is Fowler-Nordheim tunneling. This process involves trapping or removing electrons from a floating-gate MOSFET to represent logical states. This physical-layer understanding is crucial for creating accurate hardware models.
- **FPGA Prototyping Best Practices** Implementing memory interface logic on FPGAs is a vital step for pre-silicon validation. The best practice is to design modular, reusable IP cores that use standardized interfaces (like Wishbone or AXI), which makes them easier to integrate into larger System-on-Chip (SoC) designs.
- **The Importance of Flash Memory** Flash memory is a foundational technology in modern electronics. Understanding its introductory concepts is the first step in designing more complex non-volatile memory systems.
- **Role in Storage Architectures** Specifically, NAND flash memory plays a significant role in the architecture of modern storage systems. Its structure and function are central to how data is stored and retrieved in a vast number of devices.
- **Focus on Efficiency** Ongoing research in this field emphasizes the development of more efficient flash memory devices. This pursuit of efficiency drives improvements in performance, power consumption, and reliability in memory technology.

Expected Outcome

- A fully functional and verified synthesizable Verilog RTL model of an EEPROM memory module.
- A comprehensive and reusable Verilog testbench for validating the EEPROM design.
- A detailed synthesis report confirming the hardware viability of the design and providing metrics on area, power, and timing.
- A well-documented, reusable IP core that can be integrated into larger digital design projects for academic or research purposes.



References

1. Bez, E. Camerlenghi, A. Modelli, and A. Visconti, "Introduction to Flash Memory," in *Proceedings of the IEEE*, vol. 91, no. 4, pp. 489-502, Apr. 2003, doi: 10.1109/JPROC.2003.811702.
2. M. A. A. Sanvido, F. R. Chu, A. Kulkarni, and R. Selinger, "NAND Flash Memory and Its Role in Storage Architectures," in *Proceedings of the IEEE*, vol. 96, no. 11, pp. 1864-1874, Nov. 2008, doi: 10.1109/JPROC.2008.2004319.
3. M. P. Kumar, A. K. Gupta, G. J. Rao, K. S. Kumar, and P. K. Vani, "An Efficient Flash Memory Devices," *2019 IEEE International Conference on Intelligent Systems and Green Technology (ICISGT)*, Visakhapatnam, India, 2019, pp. 49-52, doi: 10.1109/ICISGT44072.2019.00026.
4. P. Pavan, R. Bez, P. Olivo, and E. Zanolini, "Flash memory cells—an overview," in *Proceedings of the IEEE*, vol. 85, no. 8, pp. 1248-1271, Aug. 1997, doi: 10.1109/5.622505.
5. J. Bergeron, *Writing Testbenches: Functional Verification of HDL Models*, Second Edition, Springer, 2003.
6. M. S. Wagh and H. M. Pande, "FPGA implementation of memory interface logic for system on chip," in *2015 International Conference on Pervasive Computing (ICPC)*, Pune, India, 2015, pp. 1-4, doi: 10.1109/PERVASIVE.2015.7087224.

Thank you